CPReady Hardware Testing, Calibration & Parameter Tuning Guide

Target Audience: Testing Team, Capstone Evaluators & Lab Assistants

Hardware Setup: ESP32 Dev Module + PCA9548A I2C Multiplexer + Dual MPU-6050 Accelerometers

Firmware Configuration File: `CPReady_Program/include/CPReadyConfig.h`

---

1. Introduction: How the System Works (In Plain English)

The CPReady vest measures how well a rescuer performs cardiopulmonary resuscitation (CPR) on a training manikin. To do this accurately, it uses **two identical motion sensors (MPU-6050s)** connected through a small traffic controller chip called the **PCA9548A Multiplexer**:

1. **Chest Sensor (MPU 1 on SD3)**: Mounted on the handpiece over the manikin's sternum. It moves up and down directly with your hands.
2. **Base / Spine Sensor (MPU 2 on SD2)**: Placed flat beneath the manikin's back. It stays still on the table or floor.
3. **The Multiplexer (PCA9548A at address `0x70`)**: Because both motion sensors have the exact same hardware address (`0x68`), connecting them directly would cause them to talk over each other. The multiplexer acts as an automatic switch, letting the ESP32 talk to the Chest sensor first, then the Base sensor, without any collision.

The Differential Motion Secret

When you push hard on a training manikin resting on a table, the table or mattress flexes slightly. If we only measured the chest sensor, table bounce would trick the system into thinking you compressed deeper than you actually did.

By subtracting the spine movement from the chest movement:

```
CODE


Actual Compression = Chest Movement - Table/Spine Movement
```

Any room vibration or mattress flexing is automatically canceled out!

---

2. Where to Click: PlatformIO & File Navigation

You do **not** need to write complex code to calibrate and tune the vest. All the adjustments are numbers in a single configuration file. Here is how to find everything in **VS Code / Antigravity IDE**:

```
CODE


+-------------------------------------------------------------------------+
| [File Explorer]                                                         |
|  v CPReady_Program                                                      |
|    > docs                                                               |
|    v include                                                            |
|        BLEReadings.h                                                    |
|        CPRTypes.h                                                       |
|      * CPReadyConfig.h  <-- ALL YOUR TUNING NUMBERS ARE IN THIS FILE!   |
|        SensorManager.h                                                  |
|    > src                                                                |
|      platformio.ini                                                     |
+-------------------------------------------------------------------------+
| Bottom Blue Bar:  [Build (✓ Checkmark)]  [Upload (➔ Right Arrow)]  [Monitor]|
+-------------------------------------------------------------------------+
```

The 3 Key Buttons on the Bottom Status Bar

Look at the very bottom edge of your screen (the blue bar):

1. **Build (✓ Checkmark icon)**:
2. Click this to verify your changes. If you made a typo or missing semicolon, it will flag it without touching the physical board.

3. **Upload (➔ Right Arrow icon)**:
4. Click this to flash the new configuration into the ESP32 via your USB cable. Make sure the ESP32 is plugged in.
5. **Serial Monitor (Plug icon or `Ctrl + Alt + S`)**:
6. Opens the live console where the ESP32 prints out real-time diagnostic messages, calibration results, and stroke depths at **115200 baud**.

The Tuning Workflow:

1. Open `CPReady_Program/include/CPReadyConfig.h`.
2. Change the number you want to test (for example, change `DEPTH_CALIBRATION_K` from `11.2f` to `12.0f`).
3. Press `Ctrl + S` to save the file.
4. Click the **Upload** (➔) button on the bottom bar.
5. Watch the Serial Monitor to test your changes live on the manikin!

---

3. Phase 1: First-Time Hardware Bench Check

Before putting the vest on the manikin, verify that the electronics talk to each other cleanly.

Step 1: Physical Sensor Orientation

- Both MPU-6050 sensor boards must be oriented with the **black square chip facing UP towards the ceiling**.
- If a sensor is upside-down, gravity will read negative, and the firmware will reject the baseline during calibration.

Step 2: Plug In & Check Startup Messages

1. Connect the ESP32 to your computer using a micro-USB cable.
2. Open the **Serial Monitor** (baud rate set to `115200`).
3. Press the small **EN / RST** button on the ESP32 to reboot it.
4. You should see output matching this:

```
TEXT


========================================
     CPReady Firmware Initializing
========================================
[I2C SCAN] Probing I2C bus (SDA=GPIO21, SCL=GPIO22, Speed=100000 Hz)...
[I2C SCAN]  -> Device detected at address 0x70 (PCA9548A I2C Multiplexer)
[I2C MUX] PCA9548A Multiplexer detected at address 0x70.
[I2C MUX]  -> Channel SD2: Found MPU6050 at 0x68
[I2C MUX]  -> Channel SD3: Found MPU6050 at 0x68
[I2C MUX] Chest sensor mapped to SD3.
[I2C MUX] Base sensor mapped to SD2.
[OK] Dual-MPU mode initialized via PCA9548A: Chest on SD3, Base on SD2 (both at 0x68).
[OK] BLE GATT Server Advertising as 'CPReady'. Ready for app connection.
```

What If It Fails?

- **Error:** `No I2C devices found`: Check your 3.3V/5V and GND power wires.
- **Error:** `PCA9548A not detected at 0x70`: Check the SDA (GPIO 21) and SCL (GPIO 22) wires connecting the ESP32 to the multiplexer board.
- **Error:** `Only 1 MPU sensor detected`: The second MPU wire is loose. The good news: our dynamic scanner checks channels 0 through 7, so if you plug MPU 2 into SD2 or SD3 instead of SD1, the firmware will find it automatically!

---

4. Phase 2: Stationary Pre-Session Calibration

Whenever hand placement is confirmed on the manikin, the mobile app triggers pre-session calibration by sending the command `"CALIBRATE"`.

What Happens During Calibration:

- The mobile app triggers the calibration sequence and displays the 2.0-second progress countdown.
- The vest measures gravity and resting baseline for **2.0 seconds** (taking 200 samples).
- When complete and stable, the mobile app confirms that the baseline is captured, enabling the *"Begin Session"* button!

Troubleshooting Calibration Failures

If someone moves the manikin or fidgets during those 2 seconds, the firmware protects data integrity by failing the calibration and printing the exact reason to the Serial Monitor:

Serial Error Message	What It Means in Plain English	Practical Fix
`[CALIB FAIL] Sample count low: X / 200`	The sensor disconnected or lost communication mid-sample.	Check wire connections or ensure the I2C speed is set to 100 kHz.
`[CALIB FAIL] Net motion spread: 0.45g > 0.30g`	The rescuer's hands were trembling or the table was bumped during the 2 seconds.	Keep hands completely still and release downward pressure until the countdown finishes.
`[CALIB FAIL] Chest gravity implausible`	The chest sensor is tilted at an extreme angle (> 45°) or upside-down.	Make sure the sensor plate sits flat on the manikin sternum, chip facing UP.
`[CALIB FAIL] Tilt/pre-pressure divergence`	The rescuer was already leaning hard on the chest before calibration finished.	Rest hands lightly on the chest without pushing down before starting calibration.

---

5. Phase 3: Kinematic Parameter Tuning (The "Ballpark" Guide)

Because manikin foam, spring stiffness, and handmade vests flex differently, you will need to tune a few parameters. **Do not aim for microscopic, laboratory-grade precision.** Real CPR training is about reliable feedback in the right ballpark!

All parameters below are located in:

`CPReady_Program/include/CPReadyConfig.h`

Parameter 1: Compression Depth Scaling (`DEPTH_CALIBRATION_K`)

- **Found at**: Line 37 of `CPReadyConfig.h`
- **Default Value**: `11.2f`
- **What It Controls**: The multiplier that converts raw acceleration measurements into stroke depth in centimeters.

How to Test:

1. Place a ruler next to the manikin chest, or compress until the manikin's internal mechanical clicker activates (standard adult clickers click at 5.0 to 5.5 cm / 2.0 to 2.2 inches).
2. Deliver 10 steady compressions to the clicker.
3. Observe the reported depth in the Serial Monitor or on the mobile app.

Symptoms of Under-Tuning vs. Over-Tuning:

- **Too Low (Under-Tuned)**: You push all the way to the manikin clicker (5.0 cm), but the app only reads 3.2 cm.
- **Too High (Over-Tuned)**: A shallow, gentle push reads 6.5 cm or flags excessive depth.

Practical Rule of Thumb:

> 💡 *"Adjust K up or down until a solid push to the clicker reads in the comfortable ballpark of **5.0 to 5.5 cm** (around two inches). If your depth reads consistently shallow, nudge K up (e.g. from 11.2 to 12.5). If it reads too deep, nudge K down (e.g. to 9.8). A discrepancy of ±2 to 3 mm is completely fine and normal."*

Parameter 2: Downstroke Trigger (`COMPRESSION_START_THRESHOLD_G`)

- **Found at**: Line 38 of `CPReadyConfig.h`
- **Default Value**: `0.45f` (units in g-force)
- **What It Controls**: How hard you have to push downward before the system recognizes that a compression stroke has started.

How to Test:

1. Rest your hands on the manikin chest in the standard CPR position.
2. Fidget, wiggle your fingers, and gently shift your weight without actually pushing down.
3. Then, deliver an intentional, deliberate downward compression.

Symptoms of Under-Tuning vs. Over-Tuning:

- **Too Low (Under-Tuned)**: Just resting your palms or adjusting your finger grip triggers "ghost" compressions.
- **Too High (Over-Tuned)**: A slow or gentle downward pump is ignored by the vest.

Practical Rule of Thumb:

> 💡 *"Adjust until resting hand fidgeting produces **zero false strokes**, but the moment you deliver an intentional downward pump, it catches it immediately."*

Parameter 3: Recoil & Leaning Detection (`RECOIL_THRESHOLD_G`)

- **Found at**: Line 39 of `CPReadyConfig.h`
- **Default Value**: `-0.35f` (units in g-force)
- **What It Controls**: Verifying that the rescuer lets the chest expand completely back up between strokes. Detects "leaning", which is a dangerous real-world rescuer mistake.

How to Test:

1. Compress 5 times while intentionally leaning your body weight on the chest (do not let the chest come all the way up).
2. Compress 5 times with clean, full release (letting the manikin spring come all the way to resting position, keeping your palms lightly kissing the chest).

Symptoms of Under-Tuning vs. Over-Tuning:

- **Too Low (e.g. `-0.15f`)**: The system is too lenient; a rescuer who leans heavily still gets credited with 100% full recoil.
- **Too High (e.g. `-0.55f`)**: The system is too strict; even when you completely release pressure, it flags "Leaning" unless you literally take your hands off the manikin.

Practical Rule of Thumb:

> 💡 *"Adjust until keeping noticeable downward pressure flags **'Recoil: Incomplete'**, but simply relaxing your weight between strokes reliably scores **'Recoil: Good'** without having to lift your hands off the chest."*

Parameter 4: Manikin Spring Bounce Lockout (`REFRACTORY_LOCKOUT_MS`)

- **Found at**: Line 83 of `CPReadyConfig.h`
- **Default Value**: `250` (milliseconds)

- **What It Controls**: A brief blind window after a stroke finishes to prevent the manikin's internal spring reverberation from counting as a second compression.

How to Test:

1. Push down firmly and snap your hands upward quickly to let the manikin spring pop back up violently.
2. Then, perform rapid compressions at the AHA upper target of 120 compressions per minute (2 strokes per second).

Symptoms of Under-Tuning vs. Over-Tuning:

- **Too Short (< 180 ms)**: A single violent compression registers as 2 strokes because of spring vibration.
- **Too Long (> 320 ms)**: When pumping fast at 120 cpm, the vest misses strokes or reports your rate as half (~60 cpm).

Practical Rule of Thumb:

> 💡 *"Adjust until a rapid snap-release never causes a double-count, while pumping smoothly at **120 beats per minute** counts every single stroke reliably. A value between **220 ms and 260 ms** is almost always ideal for adult manikins."*

Parameter 5: Hand Vibration Smoothing (`LOWPASS_FILTER_ALPHA`)

- **Found at**: Line 82 of `CPReadyConfig.h`
- **Default Value**: `0.25f`
- **What It Controls**: How aggressively the software filters out jerky hand tremors and table rattles.

Practical Rule of Thumb:

> 💡 *"Keep this at `0.25f`. If live numbers jump around wildly from slight finger twitches, lower it slightly to `0.20f`. If the depth numbers feel sluggish or delayed behind your physical hands, raise it to `0.30f`."*

---

6. Phase 4: Audio Coaching & Voice Prompt Tuning

The smartphone speaker delivers voice prompts (*"Push harder"*, *"Release completely"*, *"Speed up"*). We want the coach to feel helpful, not annoying!

These settings are in `CPReadyConfig.h` lines 54–58:

- `AUDIO_PROMPT_WINDOW_MS = 3500;` (Wait 3.5 seconds of sustained errors before speaking)
- `AUDIO_PROMPT_CONSECUTIVE_ERR = 6;` (Require at least 6 bad strokes in a row)
- `AUDIO_PROMPT_COOLDOWN_MS = 5000;` (Wait 5.0 seconds of silence between tips)

Practical Rule of Thumb:

> 💡 *"Adjust these until making 1 or 2 shallow strokes by mistake **does not trigger nagging**, but doing several seconds of consistently bad compressions gives a clear, timely prompt. If the voice talks too much, raise `AUDIO_PROMPT_COOLDOWN_MS` to 6000 or 7000 ms."*

---

7. Parameter Master Reference Cheat Sheet

Keep this quick-reference table handy at your test bench:

Parameter Name	Line	Default	Safe Range	If Too Low / Under-Tuned	If Too High / Over-Tuned
`DEPTH_CALIBRATION_K`	37	`11.2f`	`8.0f – 15.0f`	Depth reads too shallow on app	Depth reads too deep / exaggerated
`COMPRESSION_START_THRESHOLD_G`	38	`0.45f`	`0.30f – 0.65f`	Ghost strokes from hand fidgeting	Gentle/slow strokes are ignored
`RECOIL_THRESHOLD_G`	39	`-0.35f`	`-0.20f – -0.50f`	Leaning rescuer falsely passes	Must lift hands off chest to pass
`REFRACTORY_LOCKOUT_MS`	83	`250 ms`	`180 – 300 ms`	Spring bounce causes double-count	Fast compressions (120 cpm) missed
`LOWPASS_FILTER_ALPHA`	82	`0.25f`	`0.18f – 0.35f`	Sluggish / lagging depth reading	Erratic / jumpy numbers from jitter
`AUDIO_PROMPT_WINDOW_MS`	55	`3500 ms`	`2500 – 6000 ms`	Phone voice nags on accidental slip	Voice takes too long to help rescuer
`AUDIO_PROMPT_COOLDOWN_MS`	57	`5000 ms`	`3000 – 8000 ms`	Voice talks constantly / overlaps	Long silence between guidance tips

8. Final Testing & Sign-Off Checklist

Before presenting your prototype or conducting a training trial, run through this 5-minute sanity check:

- ☐ **1. Sensor Flatness**: Both MPU chips face UP; the chest sensor sits flush on the manikin sternum and the base sensor sits flat beneath the back.
- ☐ **2. Multiplexer Detection**: Open Serial Monitor; confirm `PCA9548A Multiplexer detected at address 0x70` and both sensors mapped cleanly (SD3 and SD2).
- ☐ **3. Stationary Baseline**: Run `"CALIBRATE"`; confirm stable countdown and calibration success with no error warnings.
- ☐ **4. Depth Verification**: Perform 10 strokes to the manikin internal clicker; confirm reported depth reads in the **5.0 to 5.5 cm** ballpark.
- ☐ **5. Leaning & Rate Verification**: Compress at a steady 100–120 cadence; confirm rate matches a phone metronome and full recoil registers consistently when downward pressure is released.